NYU, Tandon School of Engineering

Bridge to CS

Homework #11

Submission Instructions

1. For this assignment, you should turn in 4 `.cpp` files:
   - Each question requires one file.
   - Name your files as follows:
     - Q1: `hw11_q1.cpp`
     - Q2: `hw11_q2.cpp`
     - Q3: `hw11_q3.cpp`
     - Q4: `hw11_q4.cpp`
   - You should test your code by using a main function. However, **do not include any main functions in your final submissions**.
2. Pay special attention to the style of your code. Indent your code correctly, choose meaningful names for your variables, define constants where needed, choose the most appropriate control flow statements, break down your solutions by defining functions, etc.
3. **Important:** All functions in this assignment must be implemented **recursively**. The core structure of each solution must be recursive (i.e., the function must call itself to solve smaller subproblems). Auxiliary loops for tasks like printing characters are acceptable.

Question 1: Recursive Triangle Printing

Give a **recursive** implementation to the following functions:

a) `void printTriangle(int n)`

This function is given a positive integer `n`, and prints a textual image of a right triangle (aligned to the left) made of `n` lines with asterisks.

Example: `printTriangle(4)` should print:

```
*
**
***
****
```

b) `void printOppositeTriangles(int n)`

This function is given a positive integer `n`, and prints a textual image of two opposite right triangles (aligned to the left) with asterisks, each containing `n` lines.

Example: `printOppositeTriangles(4)` should print:

```
****
***
**
*
*
**
***
****
```

c) `void printRuler(int n)`

This function is given a positive integer `n`, and prints a vertical ruler of $2^n - 1$ lines. Each line contains '-' marks as follows:

- The line in the middle ($\frac{1}{2}$) of the ruler contains `n` '-' marks
- The lines at the middle of each half ($\frac{1}{4}$ and $\frac{3}{4}$) of the ruler contains `(n-1)` '-' marks
- The lines at the $\frac{1}{8}$, $\frac{3}{8}$, $\frac{5}{8}$ and $\frac{7}{8}$ of the ruler contains `(n-2)` '-' marks
- And so on...
- The lines at the $\frac{1}{2^n}$, $\frac{3}{2^n}$, $\frac{5}{2^n}$, ..., $\frac{2^n-1}{2^n}$ of the ruler contains 1 '-' mark

Example: `printRuler(4)` should print:

```
-
--
-
---
-
--
-
----
-
--
-
---
-
--
-
```

Hints:

(a) When finding `printRuler(4)`, try to think first what `printRuler(3)` does, and how you can use it to print a ruler of size 4.

(b) You may want to have more than one recursive call.

Question 2: Recursive Array Functions

Give a **recursive** implementation to the following functions:

a) `int sumOfSquares(int arr[], int arrSize)`

This function is given `arr`, an array of integers, and its logical size, `arrSize`. When called, it returns the sum of the squares of each of the values in `arr`.

Example: If `arr` is an array containing `[2, -1, 3, 10]`, calling `sumOfSquares(arr, 4)` will return `114` (since $2^2 + (-1)^2 + 3^2 + 10^2 = 114$).

```
114
```

b) `bool isSorted(int arr[], int arrSize)`

This function is given `arr`, an array of integers, and its logical size, `arrSize`. When called, it should return `true` if the elements in `arr` are sorted in an ascending order, or `false` if they are not.

Examples:

- `isSorted({1, 2, 3, 4, 5}, 5)` returns `true`
- `isSorted({1, 3, 2, 4, 5}, 5)` returns `false`
- `isSorted({5}, 1)` returns `true` (single element is sorted)
- `isSorted({1, 1, 2, 2}, 4)` returns `true` (equal elements are allowed)

Question 3: Two Recursive Versions of minInArray

Write two **recursive** versions of the function `minInArray`. The function will be given a sequence of elements and should return the minimum value in that sequence. The two versions differ from one another in the technique used to pass the sequence to the function.

a) **Version 1:** `int minInArray1(int arr[], int arrSize)`

Here, the function is given `arr`, an array of integers, and its logical size, `arrSize`. The function should find the minimum value out of all the elements in positions: $0, 1, 2, \ldots,$ `arrSize`-1.

b) **Version 2:** `int minInArray2(int arr[], int low, int high)`

Here, the function is given `arr`, an array of integers, and two additional indices: `low` and `high` (`low` $\leq$ `high`), which indicate the range of indices that need to be considered. The function should find the minimum value out of all the elements in positions: `low`, `low` $+$ $1, \ldots,$ `high`.

Example usage:

```
int arr[10] = {9, -2, 14, 12, 3, 6, 2, 1, -9, 15};

minInArray1(arr, 10);       // returns -9
minInArray2(arr, 0, 9);     // returns -9
minInArray2(arr, 2, 5);     // returns 3
minInArray1(arr+2, 4);      // returns 3 (arr+2 points to element at
   index 2)
```

```
-9 -9
3 3
```

Question 4: Jump It Game

The game of "Jump It" consists of a board with `n` positive integers in a row, except for the first column, which always contains zero. These numbers represent the cost to enter each column. Here is a sample game board where `n` is 6:

0	3	80	6	57	10

The object of the game is to move from the first column to the last column with the lowest total cost.
The number in each column represents the cost to enter that column. You always start the game in the first column and have two types of moves:

- Move to the adjacent column (one step forward)
- Jump over the adjacent column to land two columns over

The cost of a game is the sum of the costs of the visited columns.
In the board shown above, there are several ways to get to the end. Starting in the first column, our cost so far is 0. We could jump to 80, then jump to 57, and then move to 10 for a total cost of $80 + 57 + 10 = 147$. However, a cheaper path would be to move to 3, jump to 6, then jump to 10, for a total cost of $3 + 6 + 10 = 19$.
Implement the following function:

```
int jumpIt(int arr[], int arrSize)
```

This function takes a game board represented as an array and its size, and returns the lowest cost to reach the end.
Note: Your function should only return the lowest cost, not the actual sequence of jumps.
Examples:

- `jumpIt({0, 3, 80, 6, 57, 10}, 6)` returns `19`
- `jumpIt({0, 1, 2, 3, 4, 5}, 6)` returns `9` (path: step to 1, jump to 3, jump to 5 $\rightarrow$ $0 + 1 + 3 + 5 = 9$)
- `jumpIt({0, 100, 1, 100, 1}, 5)` returns `2` (path: jump to index 2, jump to index 4 $\rightarrow$ $0 + 1 + 1 = 2$)

```
19
9
2
```